GRAPH THEORY AND APPLICATIONS

INSTRUCTOR NAME: Marjan Alavi

TOPIC: Autonomous Street-Cleaning Vehicle — Undirected Chinese Postman Problem

Instructions. Answer all parts (a)–(h). Show your reasoning and calculations. Include the requested weighted graph, adjacency matrix, Python code, route visualization, and comparison table. The bonus question is optional. Submit your completed work through Avenue to Learn.

Problem Statement

An autonomous electric street-cleaning vehicle must clean every road of a residential neighbourhood, starting and finishing at charging station A. The road network contains eight intersections A–H and the following two-way roads.

Road	Length (m)	Direction	Road	Length (m)	Direction
A–B	120	Two-way	D–E	110	Two-way
A–D	150	Two-way	E–F	140	Two-way
B–C	100	Two-way	E–G	170	Two-way
B–E	180	Two-way	F–H	120	Two-way
C–D	130	Two-way	G–H	150	Two-way
C–F	160	Two-way	D–G	200	Two-way

Operating Constraints

- Clean every road at least once and return to charging station A.
- Energy consumption: 0.25 kWh/km.
- Usable battery capacity: 1.0 kWh.
- Recharging is available only at A.
- Minimize total travel distance.

Part A — Graph Representation and Route Optimization

(a) Construct the undirected weighted graph and identify the vertices, edges, and weights.

(b) Give its weighted adjacency matrix M and a Python method for calculating total road length.

(c) Determine whether an Eulerian circuit exists; identify all odd-degree vertices.

(d) Solve the undirected Chinese Postman Problem (CPP): determine the minimum duplicated distance, the duplicated roads, and an optimal closed route.

Part A — Graph Representation and Route Optimization (continued)

(e) Calculate the maximum cleaning efficiency, using

$$\text{Efficiency} = \frac{\text{Required cleaning length}}{\text{Total length actually travelled}} \times 100\%.$$

Part B — Algorithm, Battery Constraint, and Network Change

(f) Write scalable Python code that takes a connected undirected weighted adjacency matrix as input and determines the odd-degree vertices, shortest paths, minimum-cost matching, an Eulerian route, and cleaning efficiency.

(g) Evaluate the battery constraint. If a single trip is infeasible, devise feasible trips starting and ending at A and compare their efficiencies.

(h) Close road E–F (it no longer needs cleaning); recompute the optimal route, total travel distance, and cleaning efficiency.

Bonus — Directed Roads

Explain how the problem changes if some roads are one-way.

Submission Checklist

- Weighted graph and weighted adjacency matrix.
- Calculations and a justified optimal cleaning route.
- Scalable Python code and a visualization of the route.
- Evaluation of the battery constraint.
- Comparison table for the original network and the network with road E–F closed.